pyALDIC User Guide

Augmented Lagrangian Digital Image Correlation
Desktop GUI Walkthrough

pyALDIC Project

October 15, 2026

Abstract

This guide walks a new user through a complete pyALDIC analysis, from first launching the GUI to exporting results. Each section maps directly to a panel or action in the application so you can follow along with the real interface open.

If you already know pyALDIC and just need a refresher on one step, jump to the relevant section via the table of contents.

Contents

1 Overview **4**
- 1.1 Three-column layout . . . 4
- 1.2 Typical session times . . . 5

2 Installing and launching **5**
- 2.1 Installation . . . 5
- 2.2 Launching the GUI . . . 7
- 2.3 What you should see first . . . 7
- 2.4 Color theme and language . . . 8

3 Loading images **8**
- 3.1 Dropping or browsing a folder . . . 8
- 3.2 The image list . . . 8
- 3.3 Navigating frames . . . 9

4 Choosing the Workflow Type **9**
- 4.1 Tracking Mode . . . 9
- 4.2 Solver . . . 9
- 4.3 Reference Update (incremental only) . . . 10
- 4.4 Which combination to pick? . . . 10

5 Drawing a Region of Interest **11**
- 5.1 Live hint . . . 11
- 5.2 The Region of Interest toolbar . . . 11
- 5.3 Drawing shapes . . . 12
- 5.4 Per-frame editing . . . 13

6 DIC Parameters **13**
6.1 Subset Size . . . 13
6.2 Subset Step . . . 14
6.3 Search Range . . . 14
6.4 Mesh Refinement . . . 14

7 Advanced parameters **15**
7.1 Initial Guess . . . 15
7.2 Starting Points workflow . . . 15
7.2.1 Multi-seed tolerance . . . 16
7.2.2 Reference-frame switches in Starting Points mode . . . 17
7.3 Auto-expand search on clipped peaks . . . 17
7.4 Mesh appearance . . . 17

8 Canvas interactions **17**
8.1 Zoom and pan . . . 17
8.2 Frame navigation . . . 18
8.3 Show on deformed frame . . . 18

9 Running the DIC **18**
9.1 The Run button . . . 18
9.2 Canceling . . . 18
9.3 Progress display . . . 18
9.4 Fatal errors . . . 19
9.5 Completion . . . 19
9.6 Tracking quality warning . . . 19

10 Viewing results **20**
10.1 FIELD section . . . 20
10.2 VISUALIZATION section . . . 20
10.3 PHYSICAL UNITS section . . . 21
10.4 Console log . . . 21

11 Strain post-processing **22**
11.1 Strain Parameters panel . . . 22
11.2 Compute Strain button . . . 24
11.3 Strain field selector . . . 24
11.4 Strain vs main window . . . 25

12 Exporting **25**
12.1 The Export dialog . . . 25
12.2 Destination folder . . . 28
12.3 Running an export . . . 28

13 Sessions — save and resume **28**
13.1 File format . . . 28
13.2 What is saved . . . 28
13.3 What is not saved . . . 29
13.4 Menu . . . 29

13.5 Including results . . . 29
13.6 Opening a session whose images moved . . . 29
13.7 Corrupt or future-version files . . . 30
13.8 Legacy sessions from pyALDIC 0.5 and earlier . . . 30

14 Post-processing analysis **30**
14.1 Placing a probe . . . 30
14.2 Selecting and editing probes . . . 31
14.3 Choosing what to plot . . . 31
14.4 Four views . . . 31
14.5 Cracks and gaps . . . 32
14.6 Load data from the testing machine . . . 32
14.7 Exporting . . . 32

15 Troubleshooting **33**
15.1 “Cannot start analysis” . . . 33
15.2 “DIC analysis failed” . . . 33
15.3 Starting Points errors . . . 33
15.4 The very first analysis takes far longer than the rest . . . 34
15.5 Slow run . . . 34
15.6 Poor results in specific regions . . . 34
15.7 Tracking quality warning . . . 35
15.8 Strain map is blank or export is all NaN . . . 35
15.9 Large rotation gives crazy strain values . . . 36
15.10 Refine brush grayed out . . . 36
15.11 Session file won’t open . . . 36
15.12 Windows: “Windows protected your PC” . . . 36
15.13 Windows: nothing happens when pyALDIC starts . . . 36
15.14 macOS: “Apple could not verify ‘pyALDIC’ is free of malware” . . . 37
15.15 macOS: “pyALDIC is damaged and can’t be opened” . . . 37
15.16 macOS: nothing happens when pyALDIC starts . . . 37

1 Overview

pyALDIC computes full-field displacement and strain from a sequence of grayscale images of a deforming specimen. The typical workflow, from left to right through the GUI, is:

1. **Load images** (a folder of frames, first frame = reference).
2. **Choose Workflow Type** — tracking mode, solver, and reference-update policy. These choices are made *before* drawing a Region of Interest because they determine which frames need one.
3. **Draw a Region of Interest** on the required frame(s).
4. **Set DIC parameters** — subset size, node spacing, search range, mesh refinement.
5. **Run the analysis.** Progress, Cancel, and console log are on the right panel.
6. **View displacement results** live in the canvas.
7. **Compute strain** in the Strain Post-Processing window (opens automatically when the run finishes).
8. **Export** numerical data, images, or animations.
9. **Save the session** (File → Save Session) — one `.aldic` file keeps parameters, Regions of Interest, view state, and the computed results, so you can reopen the project later without recomputing.

Note

Video tutorials. A full end-to-end walkthrough of this workflow is available on YouTube (https://www.youtube.com/watch?v=aLqDyM3tQII, English; https://www.youtube.com/watch?v=27TtJEMefNA, Chinese) and on Bilibili (https://www.bilibili.com/video/BV1dBjm6zEnQ, P1 Chinese / P2 English).

Note

pyALDIC does **not** require you to pre-compute calibration, stereo matching, or any 3D reconstruction. It is strictly 2D DIC (one camera, planar specimen). If the specimen moves out of plane or the illumination changes significantly between frames, results will degrade and the GUI will warn you.

1.1 Three-column layout

The main window has three columns:

Left sidebar
: Images panel, *Workflow Type* section, *Region of Interest* section, *Parameters* section, and a collapsible *Advanced* section.

Center canvas
: Zoom / pan viewport showing the current frame with overlays (ROI, mesh, displacement field).

Right sidebar
: Run button, progress bar, field selector (disp_u, disp_v, etc.), visualization controls (colormap, range, opacity), physical units, and console log.

1.2 Typical session times

On a modern laptop (8-core CPU, 16 GB RAM):

Image size	Grid spacing	Time per frame
256×256	step 8	$\sim$ 0.2 s
512×512	step 8	$\sim$ 1 s
1024×1024	step 4	$\sim$ 3 s

Times scale linearly with the number of mesh nodes. The *Local DIC* solver is roughly 3× faster than *AL-DIC*.

2 Installing and launching

2.1 Installation

Pick the route that fits your computer. The two downloads need no Python at all, and every route gives you the identical application.

You have	**Use**
Windows 10 or 11	Option A, the installer — or Option A′, the portable zip, where you may not install software
A Mac with Apple silicon (M1 or later), macOS 14 or later	Option B, the disk image
Python 3.10–3.14, on any system	Option C, `pip`

All downloads are on the release page, https://github.com/zachtong/pyALDIC/releases/latest.

Option A: Windows installer

1. Download `pyALDIC-Windows-Setup.exe` from the latest release.
2. Double-click it. It installs for your own user account, so it asks for no administrator password and works on a lab computer too. Keep or untick the desktop shortcut and click **Install**.
3. Start **pyALDIC** from the Start menu or the desktop.

The installer also makes `.aldic` session files open in pyALDIC when double-clicked. To remove it: **Settings** → **Apps** → **Installed apps** → **pyALDIC** → **Uninstall**.

Note

pyALDIC is not code-signed, so Windows checks with you once. Your browser may say the file *isn't commonly downloaded*: choose **Keep** (in Edge: ··· → **Keep** → **Keep anyway**). SmartScreen may then show *"Windows protected your PC"*: choose **More info**, then **Run anyway**. This is SmartScreen reporting that the file is not yet widely downloaded, not a virus detection.

Option A′: Windows portable zip

For computers where you may not install software:

1. Download `pyALDIC-Windows-Portable.zip`.
2. Right-click it and choose **Extract All**. Run it only after extracting: opened from inside the zip, the program cannot find the files it needs.
3. Double-click `pyALDIC.exe` in the extracted folder.

Keep the folder together — move or copy the whole folder rather than the `.exe` alone. Beside `pyALDIC.exe` sits `pyALDIC-console.exe`: the same application with a console window attached, for when you need to see an error message that the normal build has nowhere to print. Both Windows options need Windows 10 (version 1703 or later) or 11, 64-bit, and about 500 MB of disk.

Option B: macOS disk image (Apple silicon)

1. Download `pyALDIC-macOS.dmg` from the latest release.
2. Open it and drag **pyALDIC** onto **Applications**.
3. Double-click pyALDIC in Applications.

Note

The first time, macOS stops it with *"Apple could not verify 'pyALDIC' is free of malware"*: pyALDIC is free, open-source software that is not notarized by Apple. Click **Done**, open **System Settings** → **Privacy & Security**, scroll down to the line about pyALDIC, click **Open Anyway** and confirm with your password. From then on it opens like any other app (again once after installing a new version). The disk image carries the same instructions in a text file beside the app.

It needs macOS 14 (Sonoma) or later on an M1 or newer Mac; Intel Macs are not supported.

Option C: From PyPI with pip (any system)

If you use Python already, install pyALDIC into a virtual environment so its dependencies cannot collide with anything else you have:

```
python -m venv pyaldic-env
pyaldic-env\Scripts\activate          # Windows
source pyaldic-env/bin/activate       # macOS / Linux
python -m pip install al-dic
python -m al_dic                      # starts the application
```

`python -m al_dic` works even when pip's script folder is not on your `PATH`; when it is, plain `al-dic` does the same. The virtual environment also avoids Homebrew Python's *externally-managed-environment* refusal on macOS. For an isolated, app-like install instead, `pipx install al-dic` or `uv tool install al-dic` give you an `al-dic` command. This is also how you get the Python API.

Option D: From a GitHub Release wheel

Useful when you cannot reach PyPI (corporate firewall, air-gapped network) or when you want to install a specific past version whose artifacts are still attached to its release page.

1. Go to https://github.com/zachtong/pyALDIC/releases.
2. Expand *Assets* on the release you want; download the `al_dic-<version>-py3-none-any.whl` file.
3. Install locally:

```
pip install ./al_dic-<version>-py3-none-any.whl
```

Dependencies are still fetched from PyPI at install time, so this method only bypasses PyPI for the `al-dic` package itself.

Option E: From source (development)

Editable install that lets you modify the code and see the effect without re-installing.

```
git clone https://github.com/zachtong/pyALDIC.git
cd pyALDIC
pip install -e ".[dev]"
```

The `.[dev]` extras include `pytest` and `pytest-xdist` for running the test suite.

2.2 Launching the GUI

- Windows installer (Option A): **pyALDIC** in the Start menu or on the desktop.
- Portable zip (Option A′): `pyALDIC.exe` in the extracted folder.
- macOS (Option B): **pyALDIC** in Applications, Launchpad or Spotlight.
- A Python install (Options C–E):

```
python -m al_dic
# or, when pip's script folder is on your PATH
al-dic
```

The window will not shrink below 1120×760. You can resize it freely above that; the three columns keep their proportions.

2.3 What you should see first

The first time the window opens:

- The left image list is empty and shows a drop zone that reads *"Drop image folder or Browse"*.
- The center canvas shows an empty dark background.
- The right sidebar shows a disabled **Run DIC Analysis** button (you cannot run without images and a Region of Interest).

Note

Shortly after the window appears, the console log reads *"Preparing compute kernels in the background"*. The solver is compiled to machine code the first time it runs and the result is cached, so this happens once per installation — pyALDIC starts it early so that it overlaps loading images and drawing a Region of Interest rather than stalling your first analysis. The interface may feel slightly less responsive for those few tens of seconds. A second message reports when it is done. Installing a new version invalidates the cache, so it happens again after an upgrade.

Tip

All user-visible text uses the full phrase *Region of Interest* instead of the DIC jargon "ROI". The 50-pixel Region column in the image list is abbreviated to "Region" to fit, with Need/Edit/Add button labels inside.

2.4 Color theme and language

Both live in the **Settings** menu, and both are remembered for the next start.

- **Settings** → **Theme**: *Dark* (the default) or *Light*. The switch is immediate: every open window, the strain window and its charts included, changes at once. The images and the field color maps are unaffected — they show data, not the interface — and figures exported for a paper are drawn light whatever the theme.
- **Settings** → **Language**: English, Simplified and Traditional Chinese, Japanese, Korean, German, French or Spanish. A new language applies after pyALDIC restarts.

3 Loading images

3.1 Dropping or browsing a folder

Click the drop zone or drag a folder of image files onto it. Every image in the folder is loaded in natural sort order. The first image is always treated as the reference frame (frame 1 in the UI, index 0 internally).

Supported formats: `.tif`, `.tiff`, `.png`, `.bmp`, `.jpg`, `.jpeg`. Grayscale is assumed; RGB inputs are converted to luminance on load.

Warning

All images in the folder must have the same dimensions. If they don't, loading stops and an error dialog tells you which image is the first mismatch.

Note

Large image sets. Frames are streamed from disk on demand rather than pre-loaded into RAM, and internal caches are bounded, so memory use stays roughly flat with the number of frames — hundreds of 4K images are fine on a 16 GB machine. Keep the folder on a fast local drive for the smoothest scrubbing.

3.2 The image list

After loading, the left sidebar shows three columns:

Column	Width	Content
#	fixed	1-based frame number
Filename	flex	file name without extension
Region	50 px	Need / Edit / Add button

The *Region* column button changes its label based on state:

Need This frame is a reference frame (per the current workflow type) and does not yet have a Region of Interest.

Edit This frame already has a Region of Interest; click to modify it.

Add This frame is *not* a required reference frame, but you can optionally add one.

Clicking any row's button activates Region of Interest editing for that specific frame; the canvas switches to show that frame's reference and the *EDITING REGION OF INTEREST FOR FRAME xx* banner appears.

3.3 Navigating frames

The frame slider at the bottom of the canvas scrubs through the sequence. Clicking a row in the image list also jumps to that frame. Keyboard: left/right arrow keys move by one frame when the canvas has focus.

4 Choosing the Workflow Type

The *Workflow Type* section is the first choice you make after loading images. It determines:

- How each frame is compared against a reference,
- Which solver (Local DIC or AL-DIC) runs,
- How often the reference frame refreshes (incremental only),
- **and therefore, which frames need a Region of Interest.**

Tip

Get this right before drawing Regions of Interest. The Region of Interest section below shows a live hint that updates whenever you change the workflow type, telling you exactly which 1-based frame numbers require a region.

4.1 Tracking Mode

Accumulative
: Every frame is compared directly against frame 1. Best for small, monotonic deformation (strain $< 5\%$, rotation $< 2°$). Fast, but fails for large cumulative motion because the reference and current subsets become visually too different.

Incremental
: Each frame is compared against a rolling reference frame. Required for rotations $> 5°$ or large cumulative strain. Displacements from consecutive segments are composed into cumulative output fields. Slightly slower than accumulative but handles arbitrary total deformation.

4.2 Solver

Local DIC
: Each subset is solved independently with IC-GN. Fast, preserves sharp local features, but sensitive to noise. Best for high- quality images and small deformations.

AL-DIC
: Augmented Lagrangian solver that couples local IC-GN with a global FEM regularizer. Smoother results, better strain accuracy, more robust in noisy or high-gradient regions. $\sim 3\times$ slower than Local DIC. Default.

When AL-DIC is selected, an indented *ADMM Iterations* spinbox appears (1–10, default 3). Higher is more regularized but slower.

4.3 Reference Update (incremental only)

Visible only when *Tracking Mode* = Incremental. Controls when the rolling reference frame refreshes:

Every Frame
Reference resets every frame. Most robust for large deformation (smallest per-step ratio). Each frame is then the reference for the next one; only frame 1 must have a Region of Interest (see the note below for frames without one).

Every N Frames
Reference resets every N frames. You need a Region of Interest on frame 1, $N+1$, $2N+1$, ... The live hint in the Region of Interest section lists the required frames.

Custom Frames
You type a comma-separated list of reference frames, e.g. `1, 6, 12, 24`. Frame 1 is always included.

Note

Frame 1 always needs a Region of Interest. Any other reference frame without one reuses frame 1's region: when you click *Run*, pyALDIC lists those frames and asks whether to continue. The reused region is exactly the one drawn on frame 1 — it is *not* moved to follow the specimen — so parts of the specimen that have moved out of it cannot be tracked reliably from that reference frame on. If the specimen moves, give every reference frame its own region that covers the specimen in that frame; *Batch Import* assigns regions to many frames at once (Section 5).

4.4 Which combination to pick?

Experiment	Tracking	Reference Update
Soft material, $< 2\%$ strain	Accumulative	—
Soft material, 2–30% strain	Incremental	Every Frame
Rigid rotation, $> 5°$	Incremental	Every Frame
Dynamic event / impact	Incremental	Every Frame
Quasi-static with known load steps	Incremental	Every N or Custom (align N with steps)

Note

When in doubt, use *Incremental + Every Frame*. It is slightly slower than accumulative but handles every deformation magnitude well. A region drawn on frame 1 alone is enough only while the specimen stays inside it; for a specimen that moves across the image, give each frame its own Region of Interest.

5 Drawing a Region of Interest

5.1 Live hint

The top of the *Region of Interest* section is a blue-accent info box that updates live as you change the workflow type. It tells you which frames should have a region:

- *Accumulative* → "Only frame 1 needs a Region of Interest. All later frames are compared against it directly."
- *Incremental + Every Frame* → "Each frame is the reference for the next one. If the specimen moves, give each frame a Region of Interest that covers the specimen in that frame (Batch Import can assign many at once); reference frames without one reuse frame 1's region after you confirm." The reused region is not moved to follow the specimen (Section 4).
- *Incremental + Every N Frames* → "Draw a Region of Interest on frames: 1, 6, 11, 16, . . . "
- *Incremental + Custom Frames* → Lists your custom reference frame indices.

5.2 The Region of Interest toolbar

Three rows of buttons, from top to bottom:

Row 1 — Shape operations

+ Add ▼
: Opens a popup menu: Polygon / Rectangle / Circle / Circle (3-point). Pick one, then draw on the canvas. The drawn region is added (OR'd) to the Region of Interest mask.

Cut ▼
: Same menu as Add, but the drawn region is subtracted (AND-NOT) from the Region of Interest mask. Use this to cut holes around features you don't want to track (bubbles, voids, damaged areas).

+ Refine ▼
: Paint brush for marking mesh-refinement zones (does *not* change the Region of Interest — it marks areas where the quadtree mesh should be subdivided for finer resolution).

Warning

The Refine brush is gated to frame 1 only. On any other frame the button renders with a dashed border and muted colors; hovering shows the explanation. This is because the brush paints in frame-1 pixel coordinates and is automatically warped forward — painting on a deformed frame would place refinement zones in the wrong material points.

Row 2 — Import / Batch Import

Import
: Load a grayscale / binary image and use it as the Region of Interest for the currently-edited frame. Any pixel above zero counts as "inside".

Batch Import
: Opens a dialog to import several mask files at once and assign them to specific frames. Useful when you have pre-segmented masks from e.g. SAM, thresholding, or a dedicated segmentation tool.
The dialog has three panels:

- **Available Masks** — files scanned from a folder you pick with *Browse*. Masks whose on-disk size does not match the loaded images are grayed out (the tooltip explains the mismatch) and excluded from every assignment path. Both 0/1 and 0/255 mask encodings are supported.
- **Frame Assignments** — only the frames the pipeline will actually consume a mask for are shown: frame 1 in accumulative mode, and every reference frame (per your Reference Update setting) in incremental mode. Non-reference frames are hidden because the solver never reads their masks.
- **Preview** — live image + mask overlay for the currently focused frame. Three view modes (image only, image + mask, mask only), an α slider 0–100%, and four overlay colors. Scroll to zoom (cursor anchored), left-drag to pan.

Assignment strategies:

- *Auto-Match by Name* — extract the last number in each mask filename and match to that frame index.
- *Assign Sequential* — pair the sorted mask list with the sorted required-frame list 1:1.
- *Assign Selected* — manual pairing. Selecting one mask plus several frames broadcasts the same mask to every selected frame (1:N). Selecting several masks plus one frame is rejected because the relationship is one-mask-per-frame only.

Row 3 — Save / Invert / Clear

Save Writes the current frame's Region of Interest mask to a PNG.

Invert
Flips inside/outside on the current frame.

Clear
Empties the current frame's Region of Interest mask. Does not delete other frames' regions.

5.3 Drawing shapes

After selecting a shape from Add or Cut:

Polygon
Click to place vertices, double-click or press Enter to close. Right-click to cancel.

Rectangle
Click-drag to define the bounding box.

Circle
Click the center, drag outward to set the radius, release to commit.

Circle (3-point)
Click three points on the circle's edge; the circle passing through them (the circumcircle) is fitted automatically. After the first two points a preview circle follows the cursor; the third click commits. Useful when you can see points on a circular feature's edge but the center is not obvious. If the three points are (nearly) collinear no circle can be formed and the action is canceled with a warning, so spread the points around the edge.

While drawing, the toolbar button for the active mode highlights. The canvas cursor changes to a crosshair. Click *Esc* or the already-highlighted button again to cancel.

5.4 Per-frame editing

Click the Region column button for any frame to jump to editing that frame's region. The canvas shows the selected frame's image (in frame-1 coordinates for accumulative mode, or whichever ref frame you're editing for incremental). A banner overlays the top of the canvas:

```
EDITING REGION OF INTEREST FOR FRAME 07
```

Changes are committed as soon as you finish a shape operation.

Regions on later reference frames (incremental)

In incremental mode, a reference frame's own Region of Interest also sets where pyALDIC measures from that frame. The measurement grid is laid out over frame 1's region. When a later reference frame's region reaches outside the bounding box of frame 1's region — a specimen that slides or rotates across the image — the pairs measured from that reference frame are solved on a grid of their own: the same node spacing, aligned with frame 1's nodes, spanning that frame's region plus one ring of nodes, as far as the image edges allow. A reference frame whose region stays inside frame 1's bounding box uses frame 1's grid. Either way, displacement and strain are reported on frame 1's nodes, so the displayed and exported fields keep the same layout.

Because the grid stretches to reach every part of the region, keep stray specks out of it: a speck far from the specimen enlarges that frame's grid and costs time and memory. With Starting Points, every separate piece of a region needs a Starting Point; Auto-place adds one where it can, but not on a speck over featureless background, and the run then stops at that frame (the frames before it are kept). Remove the speck from that frame's region and run again. A reference frame without a region of its own reuses frame 1's region as drawn (you confirm this when you click *Run*), so its grid stays frame 1's. If the specimen moves, draw a region on each reference frame here, or assign them all at once with *Batch Import*.

6 DIC Parameters

The *Parameters* section of the left sidebar holds the subset- and mesh-related choices. Change any of these and the downstream pipeline will recompute when you press Run.

6.1 Subset Size

The IC-GN correlation window, in pixels. Displayed as an odd number (e.g. 41), stored internally as the even half-width (40).

Value	When to use
21–31	Fine / dense speckle, small strain. Best spatial resolution.
41 (default)	General speckle patterns with $\sim$ 3–5 px particles.
51–81	Noisy images, coarse speckle, or low-contrast regions. Lower spatial resolution.

Larger subsets average over more pixels, which improves signal-to- noise but smooths out local features. Rule of thumb: each subset should contain 5–10 speckle particles.

6.2 Subset Step

Node spacing in pixels, restricted to powers of two: `4, 8, 16, 32, 64`. Smaller step means more mesh nodes and a denser displacement field.

Value	Trade-off
4	Densest grid, very slow. Use only for $\leq 512^2$ images.
8 (default for 512^2)	Good balance.
16 (default for 1024^2)	Standard choice for large images.
32–64	Very fast, coarse. Only for quick previews.

Tip

Total mesh nodes grow as (ROI area)/(step)^2. Halving the step quadruples the node count and roughly quadruples runtime. Start coarse, iterate on Region of Interest and parameters, then refine the step only when you are happy with the setup.

6.3 Search Range

Maximum per-frame displacement (in pixels) that the FFT initial search can detect. If the real displacement exceeds this, the FFT returns a clipped peak and pyALDIC auto-expands the search radius (2× each retry up to image half-size) — but this costs time.

Value	When to use
8–20 (default 20)	Typical quasi-static experiments.
40–60	Larger translations, moderate rotations in incremental mode.
80+	Use only if you know the motion is huge. Diagnose before setting high: a large search range also means more chance of picking wrong NCC peaks in repetitive textures.

Warning

If you see `FFT search: N nodes have peaks at search boundary` in the console log, the initial search range is too small. Either raise it manually or let Auto-expand (in Advanced, on by default) handle it.

6.4 Mesh Refinement

Two independent checkboxes below the search range control quadtree refinement of the mesh:

Refine Inner Boundary
: Subdivide mesh elements near *holes* inside the Region of Interest — e.g. around a bubble, void, or cut-out region.

Refine Outer Boundary
: Subdivide mesh elements near the outer edge of the Region of Interest, where boundary effects matter most.

When either box is checked (or the Refine brush has been used on frame 1), the *Refinement Level* dropdown activates:

Level	Label	Min element size
1	Light	step/2
2	Medium	step/4
3	Heavy	step/8
4	Extra Heavy	step/16
5	Ultra	step/32

Available levels depend on subset size and subset step so the min element size stays reasonable (minimum 2 px element). A live readout below the dropdown tells you the current minimum element size.

7 Advanced parameters

The bottom collapsible section on the left sidebar holds parameters that most users leave at their defaults. Expand it only if you are debugging convergence or doing cutting-edge experiments.

7.1 Initial Guess

Controls how IC-GN gets its starting displacement for each frame. It is a section of its own on the left sidebar, *INITIAL GUESS* (collapsed by default), right below *WORKFLOW TYPE*; it is described here because most users leave it at its default. Five mutually-exclusive options:

Previous frame (default, fastest)
: Warm-start from the previous frame's converged result. Best for smooth, slow motion. Automatically selected when Tracking Mode is Accumulative.

FFT when reference frame updates
: Run an FFT cross-correlation whenever a new reference frame is adopted; warm-start within each segment. This is the default for Incremental mode.

FFT every frame (safest)
: Full FFT every frame. Eliminates warm-start error propagation across frames, but slowest. Use for unpredictable motion (oscillation, impact, random).

Reset FFT every N frames
: Full FFT every N frames, warm-start in between. Bounds the worst-case warm-start error to N frames.

Starting Points (large displacement / cracks)
: Place one or more Starting Points manually on the canvas. Each point is matched to its counterpart in the deformed frame using a focused single-point cross-correlation, then the displacement field is propagated along mesh neighbors using an F-aware (first-order) extrapolation scheme. See Section 7.2 for the workflow.

7.2 Starting Points workflow

Use this mode when the ordinary FFT search struggles:

- **Large inter-frame displacement** (more than $\sim$ 50 pixels). FFT search grows quadratically with window size; single- point NCC scales linearly, so the whole-frame cost drops substantially in practice.
- **Discontinuous fields** (cracks, shear bands). The FFT peak picker does not know about mesh topology and can select the wrong side of a crack; propagation respects mesh connectivity and never crosses a mask hole.

Workflow:

1. Load images and set the Region of Interest normally.
2. In the *INITIAL GUESS* section, select the *Starting Points* radio button. Below it, a sub-panel appears with three buttons: **Place Starting Points**, **Auto-place** and **Clear**.
3. Every connected mask region that still needs a Starting Point is overlaid in red on the canvas. You must place at least one Starting Point inside each red region; a region with at least one point loses its red overlay. The *Run* button stays disabled while any region is red.
4. *Place Starting Points*: enters placement mode. Left-click inside a region to drop a point; the cursor snaps to the nearest mesh node (shown as a small circle). Right-click on a point to remove it. Press Esc to exit the mode. A banner above the canvas shows how many regions still need a Starting Point.
5. *Auto-place*: scans each region and picks the node with the highest cross-correlation peak. Convenient for dense meshes; you can still remove or replace individual points afterwards.
6. The "Search Range" field in PARAMETERS is relabelled *Starting Point Search* in this mode. It only controls the seed bootstrap search radius, which auto-expands if the peak is clipped. Other nodes use F-aware propagation, not per-node NCC.

If the pipeline cannot run your Starting Points — for instance, because one of them falls in a low-texture region or is too close to a mask boundary — the run stops at that frame, keeps the frames before it, and a dialog explains what happened and suggests actions (move the point, remove small separate pieces of the Region of Interest, or choose *FFT (cross-correlation)*). See Section 15.

7.2.1 Multi-seed tolerance

When you place several Starting Points, they act as redundant anchors. In a later frame of an accumulative run, an individual seed's local template can become a poor match for the deformed image (for example because the material around it rotated or stretched significantly). pyALDIC handles this gracefully:

- Each Starting Point is bootstrapped *independently*.
- A seed whose correlation falls below the NCC threshold, or whose IC-GN solve diverges, is silently *dropped* for that frame. A short warning is logged.
- The run continues as long as every connected region still has at least one working seed among the survivors.
- If every Starting Point in a region fails, the same auto-placement algorithm used by the *Auto-place* button is invoked on that region. If it finds a viable node, the run continues with the replacement.
- Only when no Starting Point *and* no auto-placement candidate can be found in a region does the run abort with a typed error.

Practical advice: when you expect large accumulative deformation, place several Starting Points in each region rather than one. Each acts as a backup when its neighbors degrade in late frames.

The NCC threshold is **0.55**, chosen to tolerate the correlation decay that accumulative deformation produces while staying well above random-noise level. The GUI does not expose it; a script can set it through `SeedSet.ncc_threshold`.

7.2.2 Reference-frame switches in Starting Points mode

When an incremental or hybrid frame schedule switches the reference frame mid-run, your Starting Points are *automatically warped* to the new reference's coordinate system using the cumulative displacement already computed. In normal use this is transparent: you place the points once on frame 0 and the pipeline tracks them across all switches.

If the warping fails — typically because the material point has moved outside the new reference's region of interest — the pipeline does not abort the run. Instead it calls the same 3-tier auto-placement algorithm used by *Auto-place* to drop fresh Starting Points on the new reference frame, logs a warning that names the affected frame and the number of new seeds placed, and continues with the run. The frame where this happened is marked in the frame navigator (see Section 10) so you can review the handoff after the run completes.

The run only aborts when the fallback *also* fails — that is, no node on the new reference frame meets the NCC floor. In that case the normal failure dialog appears.

7.3 Auto-expand search on clipped peaks

Checkbox, on by default. When the FFT NCC peak lands on the edge of the search window (indicating the true displacement is larger), the pipeline automatically retries with 2× search range up to six times, capped at $\min(H, W)/2$. Leave this on unless you are debugging FFT behavior.

7.4 Mesh appearance

Cosmetic only. Choose the color and line width of the mesh overlay drawn on the canvas (does not affect computation).

8 Canvas interactions

The center of the window is a QGraphicsView-based canvas with layered overlays. From bottom to top:

1. Background image (reference or deformed).
2. Mesh overlay (when enabled).
3. Region of Interest outline.
4. Field overlay (post-run, color-mapped displacement or strain).
5. Refine-brush overlay (cyan, frame 1 only).

8.1 Zoom and pan

Mouse wheel
: Zoom in / out at the cursor position.

Fit
: Button at the top of the canvas toolbar — zoom to fit the full image in the viewport.

100%
: Zoom to 1:1 pixel ratio.

+ / −
: Buttons for discrete zoom steps.

Middle-click drag
: Pan the view.

Left-click drag in empty area
: Pan the view (if no drawing tool is active).

8.2 Frame navigation

The frame slider at the bottom of the canvas scrubs through the sequence. The label to the left shows the current frame number and total count.

A small play/pause button next to the slider auto-advances at a configurable frame rate — useful for reviewing displacement fields after a run.

8.3 Show on deformed frame

This toggle lives in the FIELD section of the right sidebar (not VISUALIZATION, because it controls *where* the field plots, i.e. geometry, rather than how it styles).

Checked (default)
: The field overlay is plotted on the currently-selected deformed frame, with node positions warped by the cumulative displacement.

Unchecked
: The field overlay is plotted on the reference frame (frame 1) with node positions un-warped. Useful for comparing an experiment back to the starting configuration.

9 Running the DIC

9.1 The Run button

The big blue **Run DIC Analysis** button at the top of the right sidebar starts the pipeline. It is disabled until:

- At least 2 images are loaded,
- Frame 1 has a non-empty Region of Interest.

If either condition fails, clicking Run (or trying to) raises a modal dialog telling you exactly what is missing.

9.2 Canceling

Below Run is a single red **Cancel** button (replaces the older Pause + Stop pair). It is enabled while the pipeline is running.

Canceling stops the current run cleanly: already-computed frames are kept, no further frames are processed, and state returns to IDLE (not DONE, so the Export button stays disabled).

9.3 Progress display

Below the Cancel button:

- Thin blue progress bar (0–100 % of the full pipeline).

- Text label with the current stage (*Section 3: FFT initial search*, *Section 6: IC-GN local solve*, *S8: Frame 5/19*, . . .).
- ELAPSED / REMAINING timestamps.

9.4 Fatal errors

The console log (bottom of the right sidebar) collects every info, warning, and error message with a timestamp. *Fatal* errors (missing images, Region of Interest too small, pipeline-side exceptions) also trigger a modal dialog. The dialog shows a plain- English summary; the full traceback stays in the console log.

Warning

The console can scroll off-screen if many info messages accumulate. Fatal errors will not be missed because of the modal dialog, but warnings can be. Serious tracking problems also open a dialog (Section 9.6); for everything else, if something looks wrong, check the console log.

9.5 Completion

When the run finishes cleanly, the Strain Post-Processing window opens automatically (see section 11). The Run button re-enables for another run; the Export button becomes active.

If the run is canceled or fails, Strain does not open automatically and Export stays disabled.

9.6 Tracking quality warning

At the end of a run, pyALDIC checks how well the nodes of frame 1's mesh were followed through the image sequence. The check only reports: it never changes a displacement. Its findings are written to the console log as warning or info lines; frame numbers in them are the ones shown in the GUI.

Nodes far from a measured node (warning)
: On some frame, at least 3 nodes — and at least 0.5% of the nodes still tracked into that frame — landed outside the area measured from their reference frame, more than one Subset Step from the nearest measured node, away from the image edge. Their displacement was copied from that node and may be wrong. This happens in incremental runs when the grid of a reference frame does not cover the specimen, typically a reference frame that reuses frame 1's Region of Interest after the specimen has moved out of it. It cannot happen in accumulative runs.

Nodes no longer tracked (warning or info)
: A node stops being tracked when it lands on pixels outside the Region of Interest, or where its pair has no measured displacement. Its displacement is NaN from that frame on, in the exported data too. This is a warning when more than 50% of the nodes that start on the specimen in frame 1 are lost by the last frame, or more than 20% between one frame and the next. A smaller loss, from 1% of those nodes, is an info line.

Nodes at the image edge (info)
: Some nodes moved to within half a Subset Size of the image edge, or out of the image, where no whole subset fits, more than one Subset Step from the nearest measured node. Their displacement there was copied from that node.

In an incremental run where some reference frames were measured on a grid of their own (Section 5), an info line also gives how many, and the node count of the largest of those grids next to frame 1's.

When a run ends with a warning finding, a **Tracking Quality Warning** dialog also opens, in front of the Strain Post-Processing window, so the warning is not lost when the console scrolls. The dialog does not block: you can keep working while it is open, and starting another run closes it. It lists suggested actions, and *Show Details...* lists each finding. Info findings alone do not open it. After a canceled run, or a run that Starting Points stopped early, findings are logged but no dialog opens. Reopening a saved session does not show them again. What each finding means for your data and what to do about it: Section 15.7.

10 Viewing results

After a run, the right sidebar populates with field-viewing controls. The canvas shows a colormapped overlay of the selected field on the background image.

10.1 FIELD section

Two toggle buttons (exclusive): **Disp U** and **Disp V**. Selected field is highlighted; clicking the other switches.

Below the toggles sit two independent controls (see Section 8):

Show on
: *Deformed frame* plots the field at the displaced node positions; *Reference frame* plots it at the original ones.

Show background image
: Untick to see the field on its own, with no speckle image behind it. The *Show on* choice still applies: the node positions differ between the two even with nothing behind them.

With the background shown, the image follows the geometry — the deformed view uses each frame's own photo, the reference view frame 0 — because anything else would put the field where the photograph is not. What replaces a hidden background is set on the export dialog's *Preview & Colorbar* tab (Section 12); the same choice is used on screen.

10.2 VISUALIZATION section

Colormap

Nine built-in maps: `jet`, `viridis`, `turbo`, `coolwarm`, `plasma`, `inferno`, `RdBu_r`, `seismic`, `black_rainbow`.

RdBu_r and *seismic* are diverging (white at zero); best for displacement that crosses zero.

black_rainbow is `jet` with its green band replaced by black, so the middle of the range reads as background rather than as the brightest part of the map. Note that the dark band sits at the middle of the *color range*, not at zero strain — use a symmetric range with it, or the black will suggest a zero that is not there.

pyALDIC remembers the colormap choice per-field. Switching from `disp_u` to `disp_v` restores whatever map you set last time for that field.

Color Range

An explicit *Auto* / *Fixed* radio pair (replacing the former lone Auto checkbox) selects the range mode:

Auto (default)
: Range is set from the 2–98 percentile of visible node values (excludes extreme outliers but uses real data), rescaled every frame.

Fixed
: Type *Min* and *Max* explicitly; the bounds are kept for every frame. Useful for reporting (all frames share one range) or when you know the physically meaningful bounds.

The same Auto / Fixed pair appears in the Strain window and in the Export dialog, so the range mode is always an explicit choice.

> **Note**
> Numeric input fields accept both decimal separators: typing `0.07` or `0,07` gives the same value regardless of your operating-system locale.

Opacity

Slider (0–100%). Default 70%. Adjusts the alpha blending between the field overlay and the background image.

10.3 PHYSICAL UNITS section

A checkbox plus two fields:

Pixel Size
: Numeric value, dropdown unit (nm, µm, mm, cm, m, inch). Converts raw pixel displacements to physical length on all displacement-family fields.

Frame Rate
: Used to convert inter-frame velocity (ΔU/ frame) to physical speed (length / s).

Strain fields are dimensionless and unaffected by these settings.

10.4 Console log

Scrollable read-only widget at the bottom of the right sidebar. Color-coded by severity:

Level	Color
info	blue / white
warning	yellow
error	red
success	green

Every line is prefixed with a `HH:MM:SS` timestamp.

11 Strain post-processing

The Strain window is a separate top-level window that opens automatically when a Run completes. It computes strain from the stored displacement field, with its own parameter panel and its own visualization state (independent of the main window).

You can also open it manually via the **Open Strain Window** button in the right sidebar.

The right column of the strain window is organized as a scrollable stack of collapsible sections. *Strain Parameters*, *Field*, and *Visualization* are expanded by default; *Physical Units* and *Log* start collapsed — click their header (triangle arrow) to expand. The window opens fitted to the available screen area (it is never larger than your desktop, even on small laptops); if the content does not fit, the right column scrolls.

11.1 Strain Parameters panel

Three editors from top to bottom:

Method

Plane fitting (default)
: Weighted moving least-squares fit of a first-order polynomial to the displacement field inside a virtual strain gauge (VSG). Gaussian weights with the VSG radius as σ. Robust, smooths local noise.

FEM nodal
: Strain from shape-function derivatives of the FEM mesh. Higher spatial resolution but more sensitive to per-node noise.

VSG size (plane fitting only)

Virtual Strain Gauge diameter in pixels. Odd integer. Default 41. Larger = more smoothing. The field reports radius internally as $(VSG - 1)/2$.

A live *Strain window* $\approx N \times N$ *nodes* readout below the control translates the pixel VSG into the number of mesh data points the plane fit spans on a uniform mesh: with node spacing (subset step) s and radius $r = (VSG - 1)/2$, a node's k-th axial neighbor lies $k\,s$ away, so $N = 2\lfloor r/s \rfloor + 1$ nodes fall on each axis — exactly the set the radius search selects. The readout updates live as the VSG or the DIC step changes. (On a refined mesh the node count varies locally, so the readout reflects the uniform-mesh case.)

Trim low-confidence edges (plane fitting only)

At nodes near the ROI boundary (or the edge of an internal hole / crack), the VSG window crosses the boundary, so the local plane fit only sees points on one side and becomes biased and unreliable. This control hides those edge nodes from the displayed and exported strain.

The coefficient α (slider 0–1, default 0.70) sets how wide a boundary band to drop: a node is trimmed when its distance to the boundary is below $\alpha\times$ VSG radius.

$\alpha = 0$ No trimming — every node is reported (legacy behavior).

$\alpha = 0.70$ **(default)**
: Trims the band where the plane-fit error is elevated; calibrated so the kept region matches where the error returns to the interior baseline.

$\alpha = 1.0$
: Strictest — trims any node whose VSG window touches the boundary.

A live *Trimmed: N nodes (M%)* readout below the slider shows how many nodes the current setting removes. By default the trimmed nodes appear as a transparent band just inside the ROI edge, on the canvas and in exports alike (but see *Fill trimmed edges* below). (FEM nodal strain is computed from element connectivity rather than a radius, so this control does not apply to it.)

Warning

If $\alpha\times$ VSG radius exceeds the Region's half-thickness, *every* node is trimmed and both the strain map and the export come out entirely blank / NaN (the readout reads 100%). Reduce the VSG size, lower α, or switch to FEM nodal — see Section 15.

Fill trimmed edges (display only)

A checkbox in the strain visualization controls (off by default). When enabled, the transparent trim band is filled back in from the reliable interior nodes — interpolated where possible and *extrapolated* out to the full ROI edge where trimming the outer ring shrank the interpolation hull — for the on-canvas view and for exported *images and animations*. It is a display convenience only: the low-confidence boundary nodes are never shown (the fill comes entirely from reliable interior nodes), and exported *data* files (`NPZ` / `MAT` / `CSV`) always keep the trimmed edge as `NaN` regardless of this toggle. Leave it off when you need the trim boundary to remain visible; note the extrapolated edge is an estimate, so treat those outermost pixels as indicative only.

The fill also recovers the band trimmed along a crack or hole *inner* edge — but each cell is filled only from nodes on *its own side* of the crack (the straight line to the source node may not cross the crack). The two crack faces are therefore filled independently and the crack itself stays a sharp transparent line; the fill never bridges it.

Crack-aware rendering

Where the ROI contains a crack or an internal hole, the field overlay no longer interpolates across it: rendering respects the same discontinuity the mesh does, so the two faces of a crack are drawn independently instead of being smeared into a smooth ramp. This is automatic (no control) and applies to both the displacement and strain overlays, on the canvas and in exported images and animations.

Reference vs. deformed geometry

Strain is total-Lagrangian, so it is always computed on the frame-0 reference mesh, but the *geometry* used to display it follows the *Show on* selector:

Reference frame
: The strain is drawn on the reference frame using *frame-0* geometry — the same first-frame ROI mask the main window's displacement overlay uses. A crack that has grown by the current frame is *not* carved into the reference view (matching the displacement).

Deformed frame
: The strain is drawn on the current frame using that frame's geometry: the current crack is carved in, edge-trim follows it, and *Fill trimmed edges* recovers it face-by-face as above.

So switching the view switches the whole geometry (mask, edge-trim, fill, crack) between first-frame and current-frame consistently.

The separate *Show background image* checkbox only removes the speckle image behind the strain; it changes nothing about which geometry is used, and nothing about what is computed.

Strain field smoothing

Gaussian smoothing of the strain tensor field *after* computation (not a displacement smoother). Dropdown with four presets and a warning glyph on the aggressive one:

Preset	σ relative to step	Behavior
Off	0	No smoothing
Light	0.5	Subtle, preserves fine features
Medium	1.0	Balanced (recommended for noisy data)
Strong ▲	2.0	Aggressive, may blur real gradients

Here "step" is the DIC node spacing (the value you set as *Subset Step* in the main window). Smoothing smaller than 0.5× step has essentially no effect because the Gaussian kernel cannot reach any neighbor node; 2× step is the upper bound of "useful" smoothing.

Strain type

The strain measure the output fields report:

Infinitesimal (default)
$\varepsilon = \frac{1}{2}(\nabla u + \nabla u^T)$. Accurate for small deformation only. *Not* zero under finite rigid rotation — it reports $\cos\theta - 1$ as false strain.

Eulerian–Almansi
$e = \frac{1}{2}(I - F^{-T}F^{-1})$. Reported in the current (deformed) configuration.

Green–Lagrangian
$E = \frac{1}{2}(F^T F - I)$. Reported in the reference configuration. **Exactly zero under rigid rotation.** Use this for any experiment with rotation $> 2°$.

11.2 Compute Strain button

Green primary button below the parameters. Click to recompute strain with the current panel settings. Runs in a background thread; a progress bar shows per-frame progress.

If you haven't changed any parameter and the strain is already computed, the stale warning label stays empty. If you change anything, a yellow *Params changed — click Compute Strain* hint appears.

11.3 Strain field selector

Below the parameters, a radio-style selector with all available fields:

- **Displacement** (shared with main window): `disp_u`, `disp_v`, `disp_magnitude`, `velocity`
- **Strain components**: `strain_exx`, `strain_eyy`, `strain_exy`
- **Derived strains**: `strain_principal_max`, `strain_principal_min`, `strain_maxshear`, `strain_von_mises` `strain_rotation` (polar-decomposition rotation angle in degrees)

Visualization controls (colormap, range, opacity) and physical-unit settings mirror the main window's but are *independent*: the strain window remembers its own colormap and range choices per field.

11.4 Strain vs main window

	Main window	Strain window
Shows	Displacement field	Strain fields (plus disp)
State	Shared AppState	Private viz state
Export button	Export Results	Export Results (same dialog)
Frame navigator	Shared	Private

12 Exporting

Both the main window and the Strain window have an **Export Results** button. Both open the *same* dialog. The difference is only which visualization preset (colormap, range, deformed toggle) seeds the dialog — whichever window you opened it from.

12.1 The Export dialog

Five tabs at the top of the dialog:

Data tab

Check boxes for numeric-data formats:

- **NPZ** — NumPy compressed archive; preferred for Python downstream.
- **MAT** — MATLAB MAT-file (scipy v5).
- **CSV** — one row per node per frame; plain text, large.

Plus a multi-select list of which fields to include (displacement, strain components, derived strains, rotation).

Variable structure

All three data formats expose the *same* variable names, so a downstream script reads identically regardless of format:

Variable	Shape	Meaning
`coordinates`	$(N, 2)$	reference node x, y (pixels)
`disp_u`, `disp_v`	(N, T)	accumulated displacement
`disp_magnitude`	(N, T)	$\sqrt{u^2 + v^2}$
`strain_*`	(N, T)	strain components / derived

N is the node count and T the number of frames. Strain columns are aligned to the displacement frames — a frame with no computed strain is left as NaN — so every matrix shares the same column count. In CSV the same fields are columns (`node_id, x_px, y_px, disp_u, ...`), one row per node, one file per frame.

Note

Displacement and strain are both reported in **world display coordinates**: the y-axis points up (opposite the image row direction), and values scale by the pixel size when physical units are enabled. This matches the on-screen fields, so `disp_v` and the shear/strain signs stay consistent within one file. The MAT file also stores `n_frames`, `n_nodes`, and a `DICpara` struct of the run parameters; a complete parameter dump is always written separately as `<prefix>_parameters_<timestamp>.json`.

Images tab

Per-field list with Export / Colormap / Auto / Min / Max / opacity (α) columns for rendering each field as JPEG / PNG / TIFF. One file per frame per enabled field. The same appearance settings can also be edited with a live preview on the Preview & Colorbar tab, which stays two-way synced with this list.

Tip

Include colorbar is ON by default. Images without a colorbar show colored patches but no scale, which is almost never useful for quantitative reports. Uncheck it only if you plan to composite a manually-placed colorbar in Illustrator / Inkscape.

Adjustable settings:

Format
: JPEG (default) / PNG / TIFF. JPEG at the default quality is visually indistinguishable for field overlays and far smaller; pick PNG or TIFF when you need lossless output.

Resolution (long edge)
: Caps the exported image's long edge (the larger of width and height; aspect ratio is kept): 512 / 768 / 1024 (default) / 1536 / 2048 px, or *Full resolution* for native size. Field detail is bounded by the mesh, so a cap is near-lossless but much smaller on disk and faster to encode.

JPEG quality
: 60–100, default 92. Ignored for PNG / TIFF.

DPI 72–600, default 150 (metadata for print sizing).

Render as
: A geometry selector and a background checkbox, matching the main window. *Deformed frame* draws the field at the displaced node positions over each frame's own image; *Reference frame* draws it at the undeformed positions over the reference image. Unticking *Show background image* exports the field alone, with the geometry choice still applying.

Hidden background
: On the *Preview & Colorbar* tab: what fills the frame when the background image is hidden — white (default), black, or transparent. Transparency is kept for PNG and TIFF; JPEG, GIF and MP4 have no alpha channel and receive white instead. With no image behind it, the field opacity becomes real transparency rather than a blend toward the fill color.

Note

Images render directly at the output resolution with a lookup-table colormap, so exports are roughly 5× faster and 35× smaller than in pyALDIC 0.5 at the default settings — a 90-frame 4K batch drops from 3.3 min / 2.8 GB to about 40 s / 80 MB.

Animation tab

Render an MP4 / GIF for each selected field, sweeping through frames. Frames are encoded one at a time (streaming), so even long 4K sequences export without a memory spike. Colorbar on by default here too. Settings:

Format
: MP4 / GIF.

Resolution (long edge)
: Same presets as the Images tab (512–2048 px or full resolution). Strongly recommended for GIF, whose file size explodes at native resolution.

FPS Playback rate; usually match the frame rate of the experiment.

Frame step
: Export every Nth frame (1 = every frame). Higher is faster and produces smaller files but looks choppier. Playback duration is preserved: FPS refers to the pre-decimation timeline.

Warning

MP4 export requires `ffmpeg` on your system PATH. If ffmpeg is missing, the dialog will switch silently to GIF and log a warning.

Report tab

Generates a multi-page PDF summarizing the run: parameters, key fields at mid and last frame, RMSE (if ground truth is available), and optional thumbnail animation.

Preview & Colorbar tab

A WYSIWYG preview of one exported frame, rendered through the *real* export path — what you see here is exactly what Export Images and Export Animation will write. Pick the field and scrub the frame slider under the preview; it re-renders as you change any setting.

Two control groups on the right:

FIELD APPEARANCE
: Colormap, Auto / Fixed range with Min / Max, and opacity for the previewed field. These are *two-way synced* with the matching row on the Images tab: editing here updates that row and vice versa. *Apply to all fields* copies the colormap, opacity, and range mode to every enabled field (each field keeps its own Min / Max).

COLORBAR STYLE
: *Position* (Right / Left / Top / Bottom), *Font size* (6–32 pt), *Font family* (sans-serif / serif / monospace), *Bar thickness* (as a fraction of the image), *Background* (Black / White), and an outward *Margin* (blank border around the exported content, as a fraction of the long edge) with its own *Margin color*. These apply to image and animation exports alike.

When physical units are enabled at the top of the dialog, the colorbar label and tick values use them (e.g. mm instead of px), both in the preview and in the exported files.

12.2 Destination folder

At the top of the dialog. Defaults to `<image_folder>/al-dic-export/<timestamp>/`. Click *Browse* to pick a different location. Exports are written inside that folder with a user-supplied prefix (default `result`).

12.3 Running an export

Click the primary **Export** button at the bottom. Progress shows per-file (data) and per-frame (images / animations). The Cancel button stops cleanly; already-written files are kept.

When the export completes, a green success message tells you how many files were written and their location. The dialog stays open so you can export more formats; click Close when done.

13 Sessions — save and resume

pyALDIC can save your *entire* project — image folder reference, parameters, Regions of Interest, view state, and (optionally) the computed results — into a single `.aldic` file. Opening the session later lands you back on the exact page you left: same frame, same field, same colormap and color ranges, with no recompute. Hours of computation survive closing the GUI.

13.1 File format

- Extension: `.aldic` — a single-file bundle containing a human-readable JSON configuration (`session.json`) plus, when results are included, a compressed NumPy archive (`results.npz`).
- Single-file portable: Regions of Interest are stored inline as base64-encoded PNG. Result arrays are deduplicated by content (per-frame meshes that are identical across an accumulative run are stored once) and written without Python pickling, so opening a bundle from an untrusted source can never execute code.
- Versioned: the `schema_version` key protects against silent misinterpretation of newer schemas. Legacy `.aldic.json` sessions from pyALDIC 0.5 and earlier still open (see below).

13.2 What is saved

- Image folder path and the ordered list of filenames.
- Every per-frame Region of Interest mask.
- Core DIC parameters: subset size, step, search range, tracking mode, reference-update policy, solver choice, refinement settings, initial-guess mode.
- Physical units (pixel size, unit, frame rate).
- View state: current frame, selected field, deformed / reference toggle, overlay opacity, mesh appearance, and each field's colormap, range mode, and bounds.
- **Computed results** (optional): displacement and strain fields plus the mesh, byte-faithfully — reopening the session needs no recompute.

13.3 What is not saved

- The images themselves — the bundle stores the folder path and filenames, never pixel data. Keep the image folder around (or re-select it after moving).
- Run state / progress / elapsed time.
- Starting Points (the seed markers of the *Starting Points* init-guess mode). Seeds reference mesh node indices, which change with the Region of Interest and parameters — re-place or Auto-place them after opening a session.

13.4 Menu

File → Save Session...
(`Ctrl+S`) Opens a file dialog. Extension `.aldic`, appended automatically. Overwrites if the file exists.

File → Open Session...
(`Ctrl+O`) Opens a file dialog. Filter: `pyALDIC Session (*.aldic *.aldic.json)`.

File → Associate .aldic files with pyALDIC...
Windows only. Registers the `.aldic` extension for the current user (no administrator rights needed) so double-clicking a session file in Explorer launches pyALDIC and opens it directly. The command line works the same way: `al-dic path/to/run.aldic`.

13.5 Including results

If results exist when you save, a prompt asks whether to include the computed results in the bundle and shows an estimate of their uncompressed size:

Yes (default)
Writes the full results. Reopening the session restores every displacement and strain field without recomputing — for long runs this is the whole point.

No Writes a small configuration-only file (parameters, Regions of Interest, view state). Ideal for sharing a setup with a colleague.

Saving and loading run on a background thread behind a progress dialog, so multi-gigabyte bundles never freeze the GUI. The progress bar tracks the actual bytes written, array by array.

13.6 Opening a session whose images moved

If the image folder path in the session no longer exists, pyALDIC does *not* fail the load. Instead it:

1. Restores parameters and Regions of Interest.
2. Logs a warning: "Image folder ... no longer exists; parameters and Regions of Interest were restored but images must be re-selected manually."
3. Leaves the image list empty.

You can then drop a new folder of the same images and the Regions of Interest will still be aligned.

Tip

This behavior is deliberate so you can share a `.aldic` bundle with a colleague whose images live at a different path. They re-drop the images; all parameters and Regions of Interest come along.

13.7 Corrupt or future-version files

A corrupt bundle (unreadable zip), malformed JSON, or an unknown `schema_version` raise a modal error dialog. The fatal- error path is used so you cannot miss the failure. The application keeps whatever state it had before the attempted load; nothing is silently replaced.

13.8 Legacy sessions from pyALDIC 0.5 and earlier

Older versions saved sessions as a plain JSON file with the extension `.aldic.json`. These still open through File → Open Session: the loader sniffs the file content (zip bundle vs. plain JSON) rather than trusting the extension. Legacy files contain no results or view state, so those parts simply restore to defaults. Saving again writes the current `.aldic` bundle format.

14 Post-processing analysis

The Strain window has two tabs. *Strain Field* is the map covered in Section 11; *Analysis* answers the other question — not "what does the field look like now" but "what happened here": a point's displacement over time, a region's mean strain, a virtual extensometer, the opening of a crack, the field along a line, and — with the testing machine's record — a stress–strain curve.

14.1 Placing a probe

The toolbar above the canvas holds seven tools; hover over one for its name. Click a tool, then click on the image:

- **Point** — one click.
- **Line** — two clicks, start and end.
- **Rectangle** and **Circle** — two clicks (opposite corners; center, then the edge).
- **Polygon** — a click per vertex, then a double-click to close.
- **Virtual extensometer** — a line whose two clicks are the gauge points. The chart switches to extensometer strain.
- **Crack gauge** — a line with one click on each side of the crack. The chart switches to crack opening.

While a tool is armed, a note at the top of the canvas says what it is waiting for; **Esc** cancels. While a line is being drawn, its length is shown next to the cursor — standards fix the gauge length L_0, so an extensometer is placed to a length, not just between two spots.

Note

The canvas always shows the *reference* image, whichever frame the field tab displays: probe coordinates are reference-frame pixels, and a marker drawn over a deformed frame would sit beside the material it measures. The probe still follows the material — displacement is stored relative to the reference, so a fixed reference position reads one material point's history.

Show field colors the image with the plotted field at the current frame, drawn exactly as the Strain Field tab draws it (the field's own colormap and range). For a gauge reading, which has no field, it shows the Strain Field tab's field. The colorbar uses the chart's strain unit.

14.2 Selecting and editing probes

With no tool armed:

- **Click** a probe to select it; its row is selected in the list and its curve is drawn thicker. Click empty space to deselect.
- **Drag** a probe to move it; drag one of the selected probe's square **handles** to reshape it (a line's ends, a rectangle's corners, a circle's center and radius, a polygon's vertices). A move of a pixel or two is taken as a click, not an edit.
- **Double-click** a probe, or press **F2**, to rename it; **Delete** removes the selected probe — from the canvas or the list.
- **Drag empty space** to pan; the wheel and the zoom buttons zoom.

Markers, labels and handles keep their size on screen at any zoom. A line's label shows its length in the current unit, and follows it live while you reshape it. The list offers **Show** (hide a probe without deleting it), the name — which becomes the curve's label and its CSV column — the color, and a **Note** saying why a probe shows gaps or is not plotted.

14.3 Choosing what to plot

A chart shows one *quantity*, and every visible probe that can produce it is plotted — a point, a line and a region can share a chart of mean ε_{yy}; a strain and a displacement never share an axis. **Plot** is either a field with a **Statistic** across the probe's samples, or a gauge reading from a line's two ends:

- **Mean**, **Median**, **Maximum**, **Minimum**, **Standard deviation** — over a line's samples or a region's area (regions are sampled area-weighted, so a small region is not decided by one node). A point plots its own value.
- **Valid fraction** — how much of the probe was reliable on each frame.
- **Extensometer strain** $(L - L_0)/L_0$ and **true strain** $\ln(L/L_0)$; **Elongation** ΔL.
- **Crack opening** — the separation of the two ends along the gauge (what a clip gauge reads); **Crack sliding** — across it; **Crack opening magnitude** — the whole separation.

Strain as shows strain as a ratio, in % or in $\mu\varepsilon$. **X axis** is the frame, the time (with a frame rate in Physical Units), or — with load data — the load or the stress. **Min. valid fraction** leaves a frame blank when fewer than that fraction of a line's or region's points are reliable (default 0.5); plot *Valid fraction* to see why.

14.4 Four views

- **Over time** — each probe's reading at every frame. The dashed line is the current frame; click the chart to jump there. The frame is shared with the Strain Field tab.
- **Along the line** — the field along the selected line (or the newest) at the current frame, over up to twelve other frames in gray.
- **Kymograph** — the same line at every frame at once: distance against frame, the value as color, in the field's own colormap and range.

- **Stress–strain** — stress against the plotted quantity, one curve per probe, the current frame ringed on each. With an extensometer it is a stress–strain curve; with a crack gauge, load against opening. It needs load data (below); without a cross-section it plots load.

14.5 Cracks and gaps

Warning

Every sample is judged on its own. Where a growing crack consumes part of a line or region, the probe keeps reading the material that remains, the legend says *crack from frame N*, and those frames get hollow markers. A gauge reads across a crack by design: an extensometer's elongation then includes the opening, and a crack gauge measures exactly that opening.
In the line views, consumed material has its own gray — a shaded span in a profile, a band in a kymograph — because it is neither a zero nor a missing measurement.

A gap in a curve is a gap in the data, never bridged. The note in the probe list says why: too few valid points, strain trimmed at an edge or a hole, a gauge end lost to a crack, or a probe placed off the measured area. A line view with nothing to show names the reason you can act on — usually the strain edge trim, which a displacement plot or less trimming avoids.

14.6 Load data from the testing machine

Load data… imports the machine's CSV. Comma, semicolon and tab separators, decimal commas, a title line and a units row under the header are all understood. Choose the load column and its unit (N or kN), then how rows meet frames:

- **By time** — the machine's time column and an **offset**: the machine's time at the reference image (if the camera started 2 s after the machine, enter 2). Frame N is at $(N-1)/\text{fps}+\text{offset}$, with the frame rate from Physical Units; the load is interpolated there, and frames outside the record have none.
- **By frame number** — a column naming each row's frame, numbered from 1 or from 0. Repeated rows for a frame are averaged.

A line at the bottom of the dialog says how many frames have a load as you change the mapping, so a wrong offset shows at once. Give the initial cross-section A_0 (mm^2) for engineering stress F/A_0 in MPa, or leave it at 0 for load only. The record is saved with the session; loading new images clears it, since it belongs to one test. **Remove Load Data** clears it by hand.

14.7 Exporting

- **Probe data (CSV)** — one row per frame for every plotted probe, each value followed by its valid fraction and a flag saying why a cell is empty. With load data, `load_N` and `stress_MPa` columns follow the frame. Displacement v is written positive *up*, as the node export writes it (the screen shows it positive down); the header records each probe's geometry, the units, the frame numbering, the run's subset, step and strain settings, and where the load came from.
- **Line data (CSV)** — in the line views: one row per sample along the line, with its distance and position, and one column per frame.
- **Chart image** — a figure for a page, not a screenshot: white, 16 × 10 cm, 300 dpi, as PNG, SVG or PDF, with text kept editable in SVG and PDF and without the frame cursor or selection.

- **Copy chart** puts that figure on the clipboard; **Copy plotted data** puts the numbers as drawn — in the units shown — as tab-separated text for a spreadsheet.

Probes, their names and colors, and the load record are saved with the session (Section 13).

15 Troubleshooting

15.1 "Cannot start analysis"

A modal dialog with one of these messages:

Need at least 2 images
: Load an image folder in the *IMAGES* section.

Define a Region of Interest first
: Draw or import one for frame 1 in the *REGION OF INTEREST* section.

The Region of Interest of frame 1 is empty
: You drew something, then clicked Cut over the whole region. Draw it again.

Region of Interest too small
: The bounding box is smaller than 2 × Subset Step. Enlarge the Region or reduce Subset Step.

15.2 "DIC analysis failed"

Shows an exception type and message. The full traceback stays in the console log — scroll up to find it. Common causes:

- Images of different sizes (should be caught at load; if you see it at Run, report a bug).
- Extremely low subset correlation (black-on-black image or no speckle). Check your input data first.

15.3 Starting Points errors

Specific to the *Starting Points* initial-guess mode. Each error dialog explains what happened, suggests actions, and ends with the pipeline's own message (*Details*); the summary here explains *why* each situation happens. During a run, every such failure stops the run at the frame concerned and keeps the frames completed before it: the dialog is then *Analysis stopped early*, it names the frame, and its details give one of the causes below.

Analysis stopped early
: A frame could not be computed from the Starting Points. Place Starting Points on well-textured speckle, away from the edges of the Region of Interest and from cracks; remove small separate pieces from the Regions of Interest (every piece needs a Starting Point, and none can be placed automatically on a piece without usable speckle); or choose *FFT (cross-correlation)* in the *INITIAL GUESS* section.

Starting Point match quality too low
: The single-point cross-correlation between reference and deformed frame at that point scored below the NCC threshold (0.55, not adjustable in the GUI). The local texture is too uniform to match reliably. Move the point into a better-speckled area, or choose *FFT (cross-correlation)*.

Starting Point local solver did not converge
: The IC-GN iteration at the point hit the max iteration count without converging. Most often caused by a point too close to the edge of the Region of Interest, or by weak texture around it. Move the point further inside the region or reduce *Subset Size*.

Starting Point flagged as unreliable
: After the frame was solved, the displacement at a Starting Point was rejected as an outlier. The frame's field was propagated from the Starting Points, so the whole frame is unreliable. Move the point to a cleaner, well-textured area, away from edges and cracks. Adding more Starting Points does not help here: any one rejected Starting Point stops the frame.

Could not carry Starting Points to the new reference frame
: On a reference-frame switch (custom or every-N-frames schedules), the stored Starting Point warped outside the new reference's ROI *and* the auto-placement fallback also found no viable node on the new reference frame (every candidate fell below the NCC floor). The pipeline normally recovers from a warp failure by auto-placing fresh Starting Points and continuing — this error only appears when the new reference itself is unusable. Place the original points well inside the ROI interior, widen the new ROI, or improve the reference-frame image quality.

Region without a Starting Point
: At least one region (a separate part of the Region of Interest) has no working Starting Point, even after placing one automatically was tried. The propagation cannot cross from one region to another — every region needs a point. Make sure every region shown in red has one, or remove small separate pieces from the Region of Interest.

15.4 The very first analysis takes far longer than the rest

Expected, once per installation. The solver is compiled to machine code on first use; on a fast workstation that is around half a minute, more on a laptop. pyALDIC starts the compilation in the background shortly after the window opens, so it usually finishes while you are still loading images — watch for *"Compute kernels ready"* in the console log.

Warning

If the interface seems frozen during that first run, wait rather than killing the process. The compiled result is only written to the cache once compilation finishes, so a forced quit means the next launch starts over.

15.5 Slow run

- Subset Step too small: halve the step → 4× node count → 4× runtime. Start with step = 16 for large images.
- AL-DIC solver: ∼ 3× slower than Local DIC. Preview with Local DIC, switch to AL-DIC for the final run.
- Search Range much larger than actual motion: every FFT call costs proportional to $(\text{search})^2$. Set it just above your expected inter-frame motion.

15.6 Poor results in specific regions

- Noisy or fuzzy edges of the Region of Interest: enable *Refine Outer Boundary* to densify mesh near the edge.

- Hole boundaries (e.g. bubble edges): enable *Refine Inner Boundary*.
- Whole regions that simply failed to converge: check the console log for "n_bad_pts" warnings. Usually means Search Range was too small or the speckle was washed out there.

15.7 Tracking quality warning

Warning or info lines in the console log after a run — and, for serious cases, the *Tracking Quality Warning* dialog (see Section 9.6 for the rules) — report nodes of frame 1's mesh that were not tracked reliably. The results are kept and can be exported; the message tells you which frame to distrust.

To see the nodes concerned, go to the frame the message names, set *Show on* to *Deformed frame* (FIELD section) and tick *Show Grid* (canvas toolbar). The mesh is drawn at the tracked node positions, and only where its nodes are still tracked and inside that frame's Region of Interest (frame 1's, for a frame without one of its own): lost nodes leave gaps, and nodes whose displacement was copied tend to lag behind the specimen. The colored field is filled in across lost nodes from their tracked neighbors, so it hides the gaps.

Nodes far from a measured node

Incremental runs only. The grid of the reference frame named in the message did not reach where these nodes went. Usually that reference frame has no Region of Interest of its own: it reuses frame 1's region as drawn, and the specimen has moved out of it. Give every reference frame its own region that covers the specimen in that frame — draw it (Section 5) or assign regions to many frames at once with *Batch Import* — and run again. A reference frame whose region reaches outside frame 1's bounding box is measured on a grid that covers it.

Nodes no longer tracked

Nodes are lost where they land on pixels outside a Region of Interest.

- Incremental: a reference frame without a region of its own reuses frame 1's region as drawn, so nodes that move out of it are lost. Give every reference frame its own region that covers the specimen in that frame (*Batch Import* assigns many at once).
- Accumulative: nodes are lost where a frame's own region (drawn or imported) does not cover the specimen. Check that each such region covers the specimen in that frame.
- A large loss on one frame (more than 20% at once): check the region of that frame and of its reference frame, for example an imported mask assigned to the wrong frame.
- Cracks and holes: a node that lands on a crack or in a hole stops being tracked on purpose, so that no displacement is carried across the gap. Losses along them are expected and usually stay below the warning thresholds.

Nodes at the image edge

The specimen came within half a Subset Size of the image edge, or left the image, where no whole subset fits, so the displacement of the nodes there was copied from the nearest measured node. Keep the specimen inside the image, clear of that band, for the whole sequence; otherwise treat the values near the edge as approximate.

15.8 Strain map is blank or export is all NaN

The displacement fields look fine, but after *Compute Strain* the strain overlay shows nothing (only the background image) and every exported strain value (NPZ / CSV / MAT / PNG) is NaN.

With the default **Plane fitting** method this means the edge-trim removed *every* node. A node is dropped when its Virtual Strain Gauge (VSG) window comes within $\alpha \times$ VSG radius of the ROI,

hole, or image boundary (see Section 11). With the default VSG size 41 px (radius 20 px) and $\alpha = 0.70$ the trim band is 14 px, so a Region thinner than about 28 px — or a normal specimen with the VSG size raised too high — has *all* of its nodes trimmed. The live *Trimmed: N nodes (M%)* readout reads 100% in this case.

Fixes (any one):

- Reduce **VSG size** so that $\alpha \times (\text{VSG} - 1)/2$ stays below the Region's half-thickness.
- Lower **Trim low-confidence edges** (α) toward 0; setting it to 0 disables trimming entirely.
- Switch **Method** to **FEM nodal**, which uses no VSG window and no edge trim.

The opposite extreme — a VSG size *below* the DIC node spacing (Subset Step) — fails differently: the plane fit then has too few neighbors, so no strain is produced and the reason is written to the *Log* section (expand it to read the message). Keep the VSG radius $\geq$ Subset Step.

15.9 Large rotation gives crazy strain values

You are probably using *Infinitesimal* strain with a rotation $> 2°$. Infinitesimal strain interprets rigid rotation as strain of magnitude $\cos\theta - 1$ (about -0.13 at 30°). Switch the Strain window's *Strain type* to **Green–Lagrangian**, which is exactly zero under any rigid rotation.

15.10 Refine brush grayed out

Brush is gated to frame 1 only. Click into the image list or use the frame slider to return to frame 1. The button re-enables automatically.

15.11 Session file won't open

The fatal-error dialog shows the specific reason:

- File corrupted or hand-edited wrong: legacy `.aldic.json` files are plain JSON and can be checked with a JSON linter; `.aldic` bundles are zip archives and should not be edited by hand.
- `Unsupported schema version`: the file was saved by a newer pyALDIC version than the one you are running. Upgrade pyALDIC.
- `Image folder ... no longer exists`: this is a warning, not a failure. Parameters and Regions of Interest still load; re-select the image folder manually.

15.12 Windows: "Windows protected your PC"

SmartScreen showing an unrecognized-publisher notice the first time you run the installer (or the portable `pyALDIC.exe`). pyALDIC is not code-signed, so Windows has no reputation record for it. Choose **More info**, then **Run anyway**. Before that, the browser may call the download *not commonly downloaded*: choose **Keep**. If your antivirus quarantines the program instead, restore it and add an exclusion — self-contained Python applications are a common false positive.

15.13 Windows: nothing happens when pyALDIC starts

- Check the log at `%LOCALAPPDATA%\pyALDIC\logs\pyALDIC.log`. Every startup writes to it, and an unhandled error is recorded there in full.

- Run `pyALDIC-console.exe` from the program folder (`%LOCALAPPDATA%\Programs\pyALDIC` for the installer). It is the identical application with a console attached, so any error that has nowhere to go in the windowed build appears there.
- Portable zip only: make sure the folder was extracted, not merely opened. Windows lets you browse inside a `.zip` and run an `.exe` from there; it will not find the files it needs.
- Portable zip only: keep the path short. The program needs about 100 characters of its own, so extracting into an already deeply nested folder can exceed the Windows path limit.

15.14 macOS: "Apple could not verify 'pyALDIC' is free of malware"

Expected, once per installation: pyALDIC is not notarized by Apple. Click **Done**, open **System Settings** → **Privacy & Security**, scroll down to the line about pyALDIC and click **Open Anyway**, then confirm with your password. The button only appears for about an hour after macOS blocked the app; if it is not there, double-click pyALDIC again and look again. From Terminal, the same permission is `xattr -dr com.apple.quarantine /Applications/pyALDIC.app`.

15.15 macOS: "pyALDIC is damaged and can't be opened"

The app's contents no longer match its signature — almost always an incomplete download, or a copy made with a tool that dropped part of the bundle. Delete it, download `pyALDIC-macOS.dmg` again, and drag the app from the disk image to Applications. Also check the Mac: the download needs Apple silicon (M1 or later) and macOS 14 or later.

15.16 macOS: nothing happens when pyALDIC starts

- Check the log at `~/Library/Application Support/pyALDIC/logs/pyALDIC.log`.
- Run the app from Terminal to see any error it prints: `/Applications/pyALDIC.app/Contents/MacOS/pyALDI`